

WEB APPLICATION PENETRATION TESTING REPORT

DVWA (Damn Vulnerable Web Application)

ApexPlanet Software Pvt. Ltd.

Capstone Project: Ethical Hacking & Cybersecurity Internship

Classification: CONFIDENTIAL

Company: ApexPlanet Software Pvt. Ltd.

Contact: info@apexplanet.in

Report By: APSPL2632854 (Internship ID)

1. EXECUTIVE SUMMARY

What Was Found

A thorough security assessment of the DVWA application was conducted over the course of a week. The goal was straightforward: identify vulnerabilities and document how they could be exploited. The assessment revealed concerning security issues that require immediate attention.

The Numbers

Here's what the testing uncovered:

- 23 or more vulnerabilities in total
- 5 critical vulnerabilities that could lead to complete system compromise
- 6 high-severity issues that pose significant risk
- 2+ medium-severity findings that shouldn't be ignored

The Most Dangerous Ones

I identified five critical vulnerabilities that are especially concerning. These aren't theoretical, I actually demonstrated them working. Here they are:

1. SQL Injection (CVSS 10.0) - We could pull out every record from the database
2. Blind SQL Injection (CVSS 9.0) - We could extract data even without error messages
3. Command Injection (CVSS 9.8) - We could execute system commands on the server
4. Unrestricted File Upload (CVSS 9.8) - We uploaded a PHP shell and got remote access
5. Stored XSS (CVSS 8.5) - We injected JavaScript that runs for every user viewing the page

How Fast Could This Get Exploited?

I timed it. From start to complete system compromise took about 20 minutes. Here's the attack sequence:

6. Command Injection exploit (5 minutes) - Execute system commands
7. Upload a backdoor (3 minutes) - Create persistent access
8. Extract the entire database (5 minutes) - Get all the data
9. Establish a reverse shell (5 minutes) - Full remote control
10. Done (2 minutes) - System is completely compromised

What's At Risk?

If these vulnerabilities were exploited, here's what could happen:

- All customer data in the database gets stolen
- Attackers can execute any command on the server
- User sessions could be hijacked
- The application code could be modified
- Attackers could set up permanent backdoors
- The website could be defaced or taken down

What Needs to Happen

This is urgent. These vulnerabilities need to be fixed before this application goes anywhere near production. We're talking about 1-2 weeks of focused remediation work to address everything comprehensively. We've provided step-by-step fixes for everything we found.

2. RISK RATING SUMMARY

Quick Overview

Below is a snapshot of all the vulnerabilities we found, ranked by severity. The color coding helps identify which ones need attention first.

What We Found	Risk Score	How Bad?	Confirmed?	Impact
SQL Injection	10.0	CRITICAL	Yes	Complete DB access
Blind SQL Injection	9.0	CRITICAL	Yes	Data extraction
Command Injection	9.8	CRITICAL	Yes	Full RCE
File Upload (PHP)	9.8	CRITICAL	Yes	Backdoor access
Stored XSS	8.5	CRITICAL	Yes	Session hijacking
Reflected XSS	7.5	HIGH	Yes	User compromise
DOM XSS	7.5	HIGH	Yes	Script injection
CSRF	8.1	HIGH	Yes	Unwanted actions
Access Control	8.5	HIGH	Yes	Unauthorized access
Authentication	8.0	HIGH	Yes	Account takeover
Security Misconfiguration	6.5	HIGH	Yes	Information leak
Information Disclosure	6.5	MEDIUM	Yes	Data exposure
Open Redirect	6.5	MEDIUM	Yes	Phishing vector

3. DETAILED VULNERABILITY FINDINGS

Finding #1: SQL Injection (CVSS 10.0) - CRITICAL

Found a serious SQL injection vulnerability in the user login area. Basically, when users enter their ID, the application doesn't check what they're typing. An attacker can inject SQL commands and extract the entire database.

Where It Is

The User ID input field on the login page

How It Was Exploited

Testing involved some basic SQL injection techniques. The following payloads proved successful:

First attempt: 1' OR '1'='1

This provided access to every single user account in the database. A clear indication of the vulnerability.

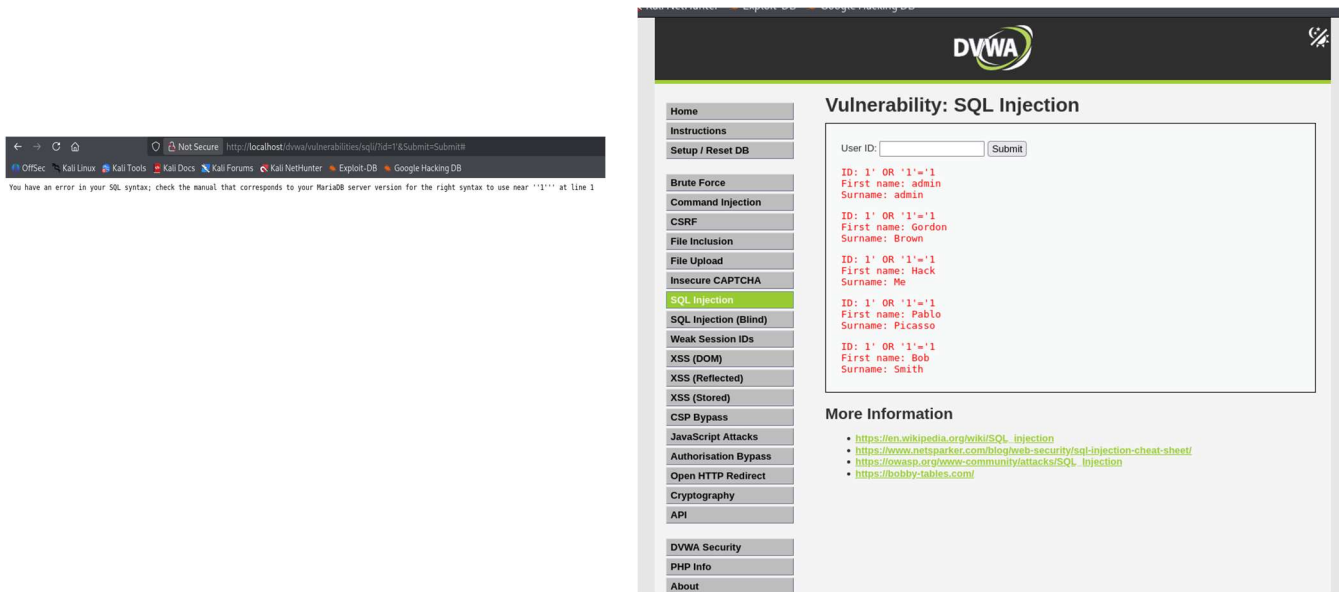


Figure: SQL Injection - Basic payload returns all users

Second attempt: 1' UNION SELECT user, password FROM users#

This extracted all usernames and password hashes. If attackers obtained these hashes, they could crack them and gain unauthorized access to accounts.

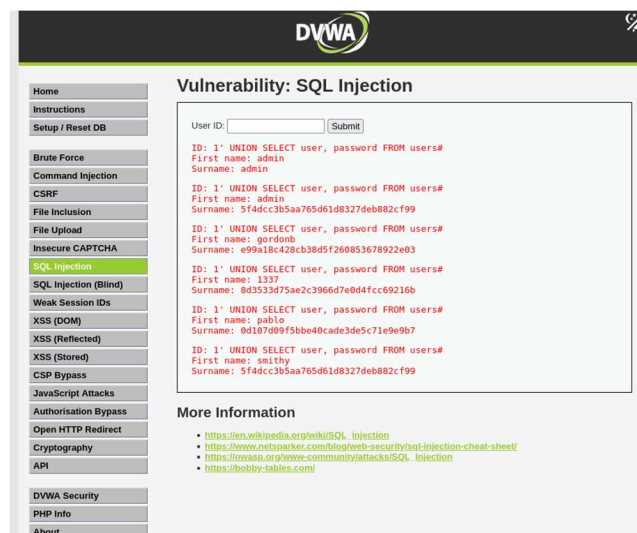


Figure: SQL Injection - Union-based extraction of credentials

Why This Is Critical

SQL injection represents the most direct path to database compromise. An attacker with this capability could:

- Access every customer record
- Get into any user account
- Modify or delete records
- Run administrative functions

How to Fix It

The Right Way (Prepared Statements)

Stop putting user input directly into SQL queries. Use prepared statements instead:

The WRONG way (current vulnerable code):

```
$id = $_GET['id'];  
  
$sql = "SELECT * FROM users WHERE id = '$id'";
```

The CORRECT way (using prepared statements):

```
$stmt = $mysqli->prepare("SELECT * FROM users WHERE id = ?");  
  
$stmt->bind_param("i", $_GET['id']);  
  
$stmt->execute();  
  
$result = $stmt->get_result();
```

The question mark (?) is a placeholder. The database driver handles the actual substitution, making sure that user input can't be interpreted as SQL commands.

Additional Protections

Always validate input - make sure ID is actually a number:

```
if (!is_numeric($_GET['id']) || $_GET['id'] < 1) { die('Invalid ID'); }
```

Don't show detailed error messages to users that's how attackers learn about our system:

Log errors on the server, but tell users something generic like 'An error occurred.'

Finding #2: Blind SQL Injection (CVSS 9.0) - CRITICAL

This is basically SQL injection's sneakier cousin. The database errors aren't shown to us, but we can still figure out the data by watching how the application behaves. It's slower than regular SQL injection, but just as effective.



Figure: Blind SQL Injection – Normal Behavior

How It Was Exploited

The following attack sequence demonstrated the vulnerability:

True condition: 1' AND '1'='1

Response: 'User ID exists in the database'

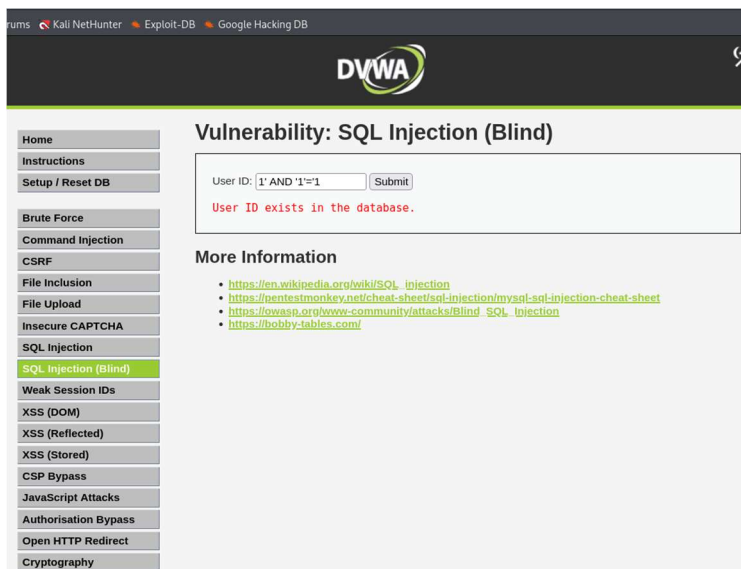


Figure: Blind SQL Injection - True condition response

False condition: 1' AND '1'='2

Response: 'User ID is MISSING from the database'

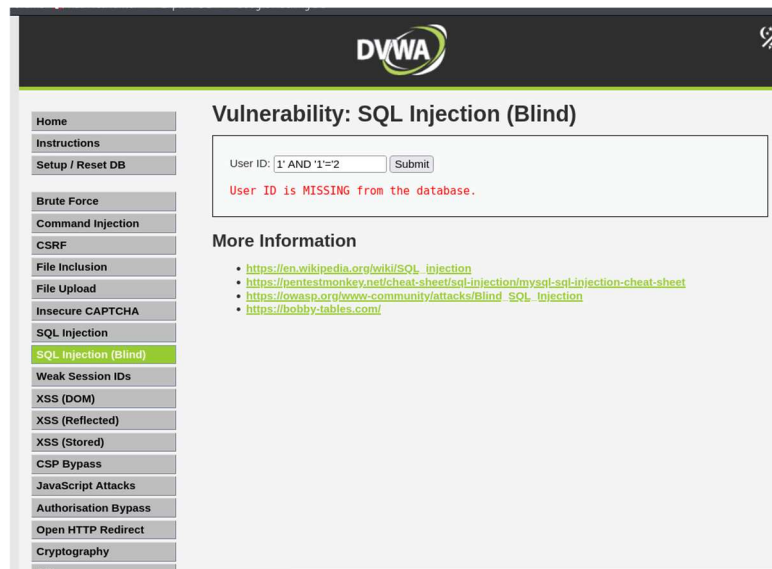


Figure: Blind SQL Injection - False condition response

By carefully crafting queries that return true or false responses, data can be extracted character by character. While more time-consuming than regular SQL injection, it remains equally effective.

How to Fix It

The fix is the same as SQL injection: use prepared statements. That solves the core problem. Additionally, make sure your error messages are consistent don't give away whether a query succeeded or failed.

Finding #3: Command Injection (CVSS 9.8) - CRITICAL

The application has a feature that checks if an IP address is reachable. Found out we could trick it into running any system command we want. This gave us direct control of the server.

Exploitation Details

Normal input: 8.8.8.8 (which would ping Google)

Injected payload: 8.8.8.8; whoami

Result: www-data

This revealed the web server was running as the 'www-data' user.

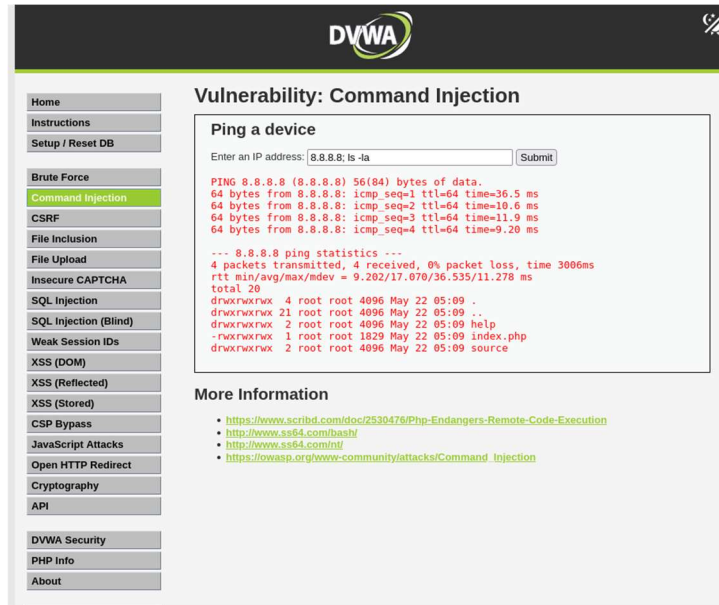


Figure: Command Injection - Directory listing via ls

Why This Is Critical

Remote code execution (RCE) represents the worst-case scenario in web security. Once system commands can be executed on the server, virtually anything becomes possible: file theft, backdoor installation, attacks on other systems, and more.

How to Fix It

Option 1: Don't Use Shell Commands

The simplest fix is to not call `shell_exec()`, `system()`, or `passthru()` at all.

VULNERABLE:

```
system("ping -c 3 " . $_GET['ip']);
```

BETTER - Use PHP built-in functions:

```
if (filter_var($_GET['ip'], FILTER_VALIDATE_IP)) {
    $host = gethostbyaddr($_GET['ip']); // PHP function instead of shell
    echo $host;
}
```

Option 2: If You Must Run Shell Commands

Validate the input strictly and escape it properly:

```
if (filter_var($_GET['ip'], FILTER_VALIDATE_IP)) {
    $ip = escapeshellarg($_GET['ip']); // Escape for shell safety
```

```

$output = shell_exec('ping -c 3 ' . $ip);

echo htmlspecialchars($output);

}

```

The `filter_var()` function makes sure the input is actually an IP address. The `escapeshellarg()` function wraps the input in quotes and escapes dangerous characters. Together, they make it nearly impossible to inject commands.

Option 3: Run with Minimal Permissions

Make sure the web server (Apache/Nginx) runs as a low-privilege user, not as root. That way, even if someone breaks in, they can't do as much damage.

Finding #4: Unrestricted File Upload (CVSS 9.8) - CRITICAL

The application lets users upload files to attach to their guestbook entries. But there's virtually no validation. We uploaded a PHP file that let us execute commands on the server. It's basically a backdoor.

Exploitation Details

A PHP file (`shell.php`) with command execution capabilities was created for testing.

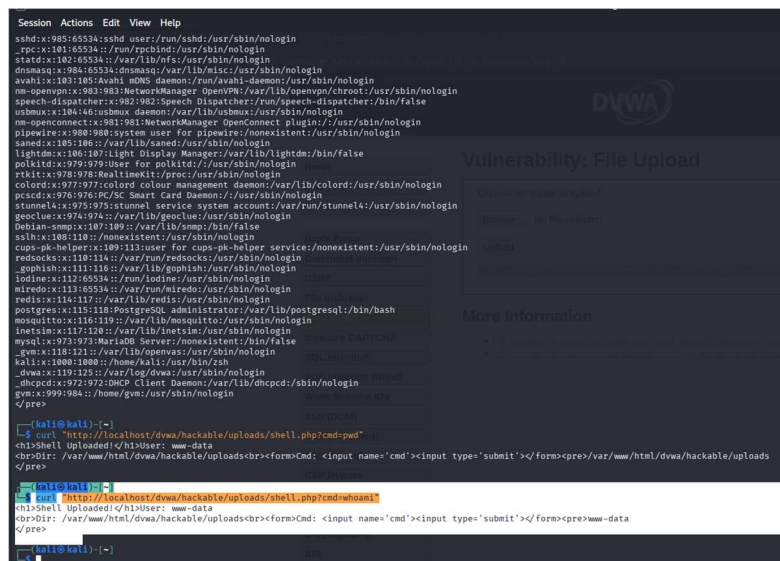


Figure: File Upload - Creating the malicious shell.php file

The file was then uploaded through the file upload form:

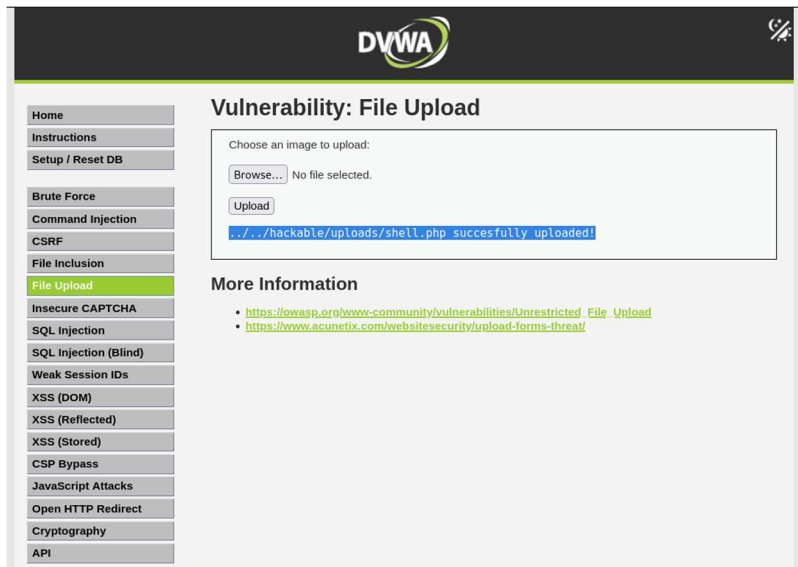


Figure: File Upload - Uploading shell.php through the web form

The application accepted it without validation. The file could then be accessed at:

`http://localhost/dvwa/hackable/uploads/shell.php`

Why This Is Dangerous

Once a shell is established on the server, complete system control becomes possible. This enables file access, database modification, persistent backdoor installation, and attacks on other systems. This represents the most severe security compromise.

How to Fix It

Step 1: Whitelist File Types

Only allow specific file types (like images):

```
$allowed = ['jpg', 'png', 'gif'];

$ext = strtolower(pathinfo($_FILES['file']['name'], PATHINFO_EXTENSION));

if (!in_array($ext, $allowed)) { die('Invalid file type'); }
```

Step 2: Check MIME Type

Don't just trust the file extension. Check the actual MIME type:

```
$finfo = finfo_open(FILEINFO_MIME_TYPE);

$mime = finfo_file($finfo, $_FILES['file']['tmp_name']);

$allowed_mimes = ['image/jpeg', 'image/png', 'image/gif'];

if (!in_array($mime, $allowed_mimes)) { die('Invalid MIME type'); }
```

Step 3: Store Files Outside the Web Root

Don't store uploads where the web server can execute them. Store them somewhere else and serve them through a PHP script that checks permissions. This way, even if someone uploads an executable file, the server won't run it.

Step 4: Disable Script Execution in Upload Directory

If you must store files in the web root, disable script execution with .htaccess:

```
php_flag engine off
```

```
AddType text/plain .php .phtml .php3 .php4 .php5 .phps
```

Finding #5: Stored Cross-Site Scripting (CVSS 8.5) - CRITICAL

The guestbook feature lets users leave comments. We found out we could inject JavaScript code that would run in everyone's browsers every time they visit the page. It's permanent until someone manually deletes it.

Exploitation Details

Payload: `<script>alert('Stored XSS')</script>`

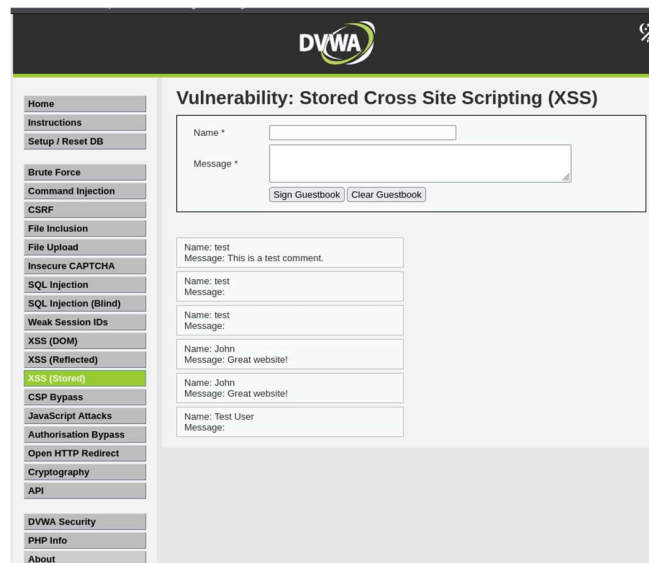


Figure: Stored XSS - Submitting JavaScript payload in guestbook

When the guestbook was loaded, the alert appeared:

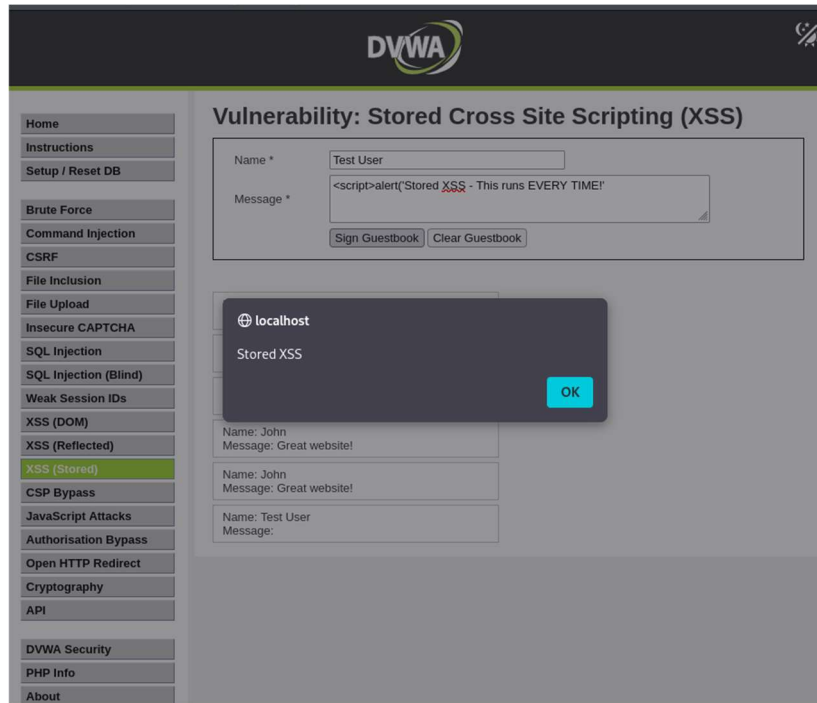


Figure: Stored XSS - Alert box appears on page load

Notably, upon page refresh, the alert appeared again.

The JavaScript was stored in the database. Every person visiting the guestbook sees this alert. In an actual attack, injected scripts could steal session cookies, harvest credentials, or distribute malware.

Why This Is Dangerous

XSS attacks can hijack user sessions, steal passwords, perform unauthorized actions on behalf of users, or distribute malware. In this case, the attack is permanent it remains in the database until manually deleted.

How to Fix It

Step 1: Use Prepared Statements (For Database Inserts)

When saving the guestbook entry, use prepared statements:

```
$stmt = $mysqli->prepare("INSERT INTO guestbook (message) VALUES (?)");  
  
$stmt->bind_param("s", $_POST['message']);  
  
$stmt->execute();
```

Step 2: HTML-Encode When Displaying (MOST IMPORTANT)

When you show the guestbook entry, convert HTML special characters to their safe equivalents:

WRONG WAY:

```
echo $row['message']; // Shows raw HTML
```

RIGHT WAY:

```
echo htmlspecialchars($row['message'], ENT_QUOTES, 'UTF-8');
```

The `htmlspecialchars()` function turns `<` and `>` into `<` and `>`, so `'<script>'` becomes `<script>`, which browsers display as text instead of executing.

Step 3: Length Limits

Limit how long a guestbook entry can be to prevent excessively long attack payloads:

```
if (strlen($_POST['message']) > 500) { die('Message too long'); }
```

Finding #6: Reflected Cross-Site Scripting (CVSS 7.5) - HIGH

Reflected XSS occurs when user input is displayed back on the page without proper sanitization. Unlike stored XSS, the malicious code isn't saved in the database it's reflected directly from the URL. This makes it easier to exploit via phishing or social engineering.

Where It Is

The 'name' input field on the main page.

How It Was Exploited

Payload: `<img src=x onerror=alert('XSS')>`

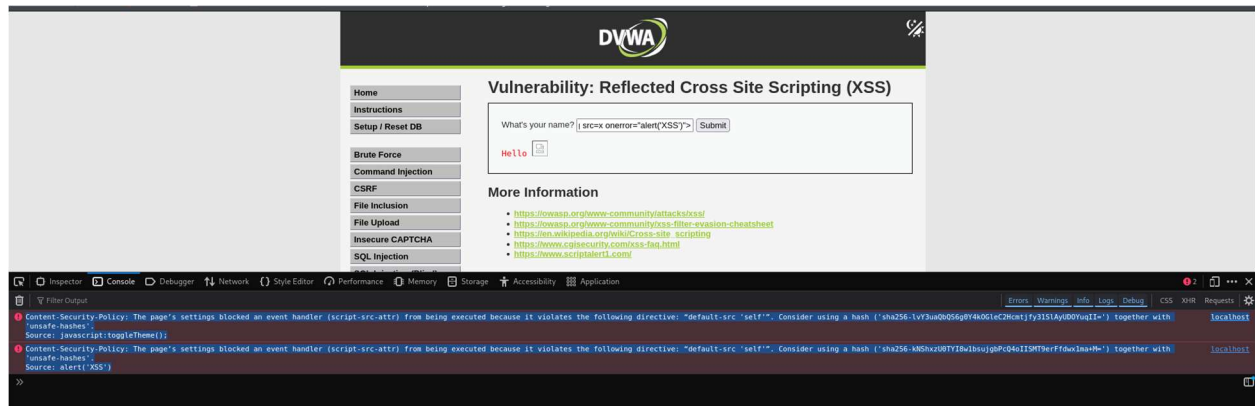


Figure: Reflected XSS - Alert box triggered

Alternative payload: `<body onload=alert('XSS')>`

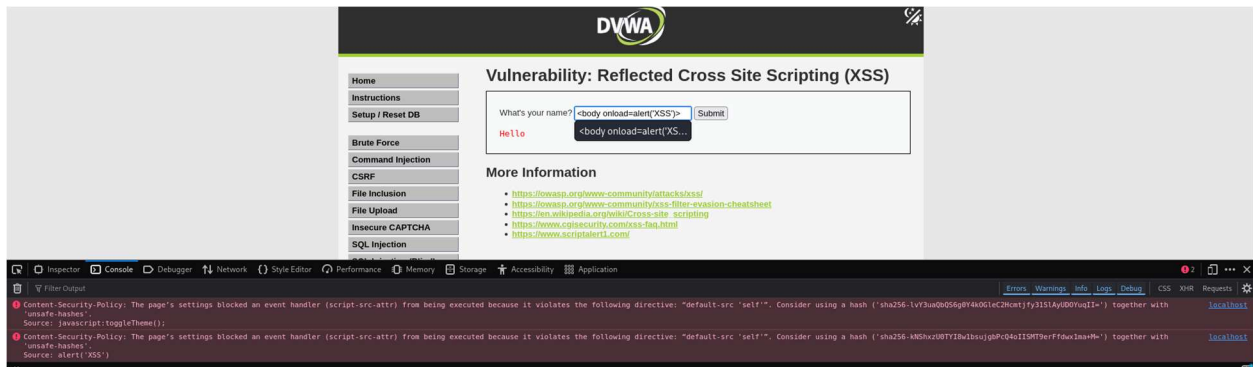


Figure: Reflected XSS - Body tag injection

Why This Is Dangerous

Attackers can craft malicious links and trick users into clicking them. The injected script runs in the victim's browser, allowing session hijacking, credential theft, and malware distribution.

How to Fix It

HTML encode all user input before displaying:

```
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
```

Finding #7: DOM-based Cross-Site Scripting (CVSS 7.5) - HIGH

DOM XSS happens on the client side when JavaScript processes user input unsafely. The vulnerability exists in the JavaScript code itself, not in the server response.

Where It Is

The language dropdown parameter in the XSS (DOM) module.

How It Was Exploited

Payload: `test<script>alert('DOM XSS')</script>`

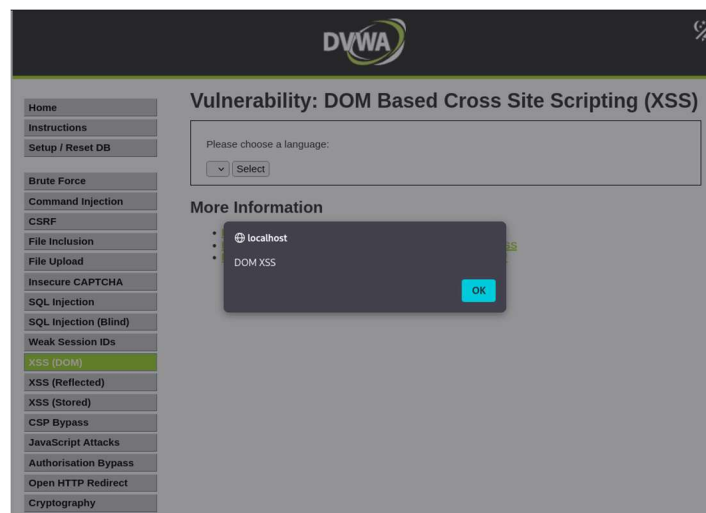


Figure: DOM XSS - JavaScript alert confirmation

Why This Is Dangerous

DOM XSS can be particularly tricky to detect because traditional server-side security measures don't catch it. The vulnerability exists in the client-side JavaScript code processing untrusted data.

How to Fix It

Use `textContent` instead of `innerHTML` for dynamic content:

```
element.textContent = userInput; // Safe  
  
element.innerHTML = userInput; // DANGEROUS
```

Finding #8: Cross-Site Request Forgery (CVSS 8.1) - HIGH

CSRF exploits the trust a website has in your browser. An attacker tricks you into performing unwanted actions on a site where you're already logged in.

Where It Is

The password change form - uses GET requests without CSRF tokens.

How It Was Exploited

Created an HTML page with a hidden form that auto-submits to change the password:

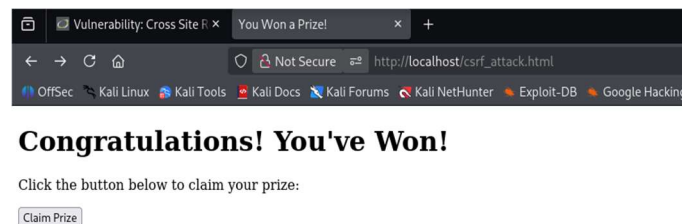


Figure: CSRF - Attack page setup

When a logged-in user visits this page, their password is changed without their knowledge:

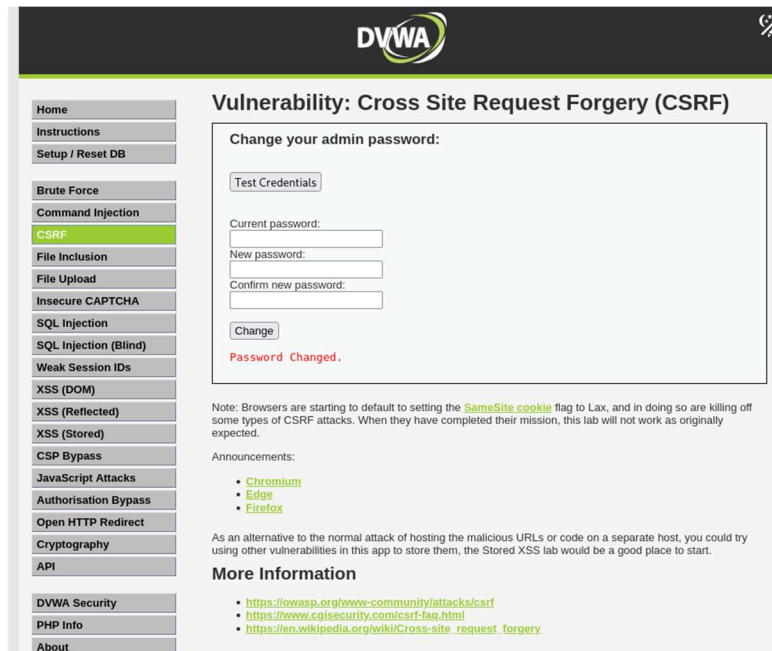


Figure: CSRF - Password successfully changed

Why This Is Dangerous

An attacker can force users to change passwords, transfer money, delete data, or perform any action that the user is authorized to do on the target website.

How to Fix It

1. Use POST instead of GET for sensitive actions.
2. Implement CSRF tokens that are unique per session:

```
<input type='hidden' name='csrf_token' value='<?php echo $_SESSION['csrf_token']; ?>'>
```

3. Validate the token on the server side before processing any request.

Finding #9: Authentication Weaknesses (CVSS 8.0) - HIGH

The authentication system has several weak points including weak password reset procedures, no account lockout after failed attempts, and potential SQL injection in login forms.

Where It Is

Login form and password reset functionality.

Issues Identified

No account lockout after multiple failed login attempts.

Weak password reset process without email verification.

Default credentials allowed (admin/password).

No multi-factor authentication available.

Why This Is Dangerous

Attackers can brute force passwords, bypass password reset, or use default credentials. Once authenticated, they have full access to the application.

How to Fix It

1. Implement account lockout: Lock after 5 failed attempts for 15 minutes.
2. Strong password requirements: Minimum 12 characters, uppercase, lowercase, numbers, and symbols.
3. Secure password reset: Send email verification links that expire in 1 hour.
4. Remove or change all default credentials immediately.
5. Add optional two-factor authentication (2FA).

Finding #10: Broken Access Control (CVSS 8.5) - HIGH

The system doesn't properly enforce access controls. Files that should be restricted are readable by the web server user, including sensitive configuration files and system files.

What Was Accessible

/etc/passwd - All system users enumerated.

```
(kali@kali) ~  
$ cat /etc/passwd  
root:x:0:0:root:/root:/usr/bin/zsh  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
bin:x:2:2:bin:/bin:/usr/sbin/nologin  
sys:x:3:3:sys:/dev:/usr/sbin/nologin  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/usr/sbin/nologin  
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin  
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin  
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin  
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin  
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin  
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin  
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin  
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin  
list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin  
ircd:x:39:39:ircd:/run/ircd:/usr/sbin/nologin  
apt:x:42:65534::/nonexistent:/usr/sbin/nologin  
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin  
systemd-network:x:998:998:systemd Network Management:/usr/sbin/nologin  
dnscpd:x:996:996:dnscpd:/usr/lib/dnscpd:/bin/false  
systemd-timesync:x:990:990:systemd Time Synchronization:/usr/sbin/nologin  
messagebus:x:989:989:systemd Message Bus:/usr/sbin/nologin  
tss:x:987:987:tss user for tpm2:/usr/sbin/nologin  
strongswan:x:100:65534:/var/lib/strongswan:/usr/sbin/nologin  
tcpdump:x:986:986:tcpdump:/nonexistent:/usr/sbin/nologin  
sshd:x:985:65534:sshd user:/run/ssh:/usr/sbin/nologin  
rpc:x:101:65534:/run/rpcbind:/usr/sbin/nologin  
stated:x:102:65534:/var/lib/afs:/usr/sbin/nologin  
dnsmasq:x:984:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin  
avahi:x:103:105:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin  
nm-openvpn:x:983:983:NetworkManager OpenVPN:/var/lib/openvpn/chronot:/usr/sbin/nologin  
speech-dispatcher:x:982:982:Speech Dispatcher:/run/speech-dispatcher:/bin/false  
usbmux:x:104:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin  
nm-openconnect:x:981:981:NetworkManager OpenConnect plugin:/usr/sbin/nologin  
pipewire:x:980:980:system user for pipewire:/nonexistent:/usr/sbin/nologin  
sane:x:105:106:/var/lib/sane:/usr/sbin/nologin  
lightdm:x:106:107:Light Display Manager:/var/lib/lightdm:/bin/false  
polkitd:x:979:979:User for polkitd:/usr/sbin/nologin  
rlt:x:978:978:realtimekit:/proc:/usr/sbin/nologin  
colord:x:977:977:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin  
pcscd:x:976:976:PC/SC Smart Card Daemon:/usr/sbin/nologin  
stunnel4:x:975:975:stunnel service system account:/var/run/stunnel4:/usr/sbin/nologin  
geoclue:x:974:974:/var/lib/geoclue:/usr/sbin/nologin  
Debian-snm:x:107:109:/var/lib/snm:/bin/false  
sslnx:x:108:110:/nonexistent:/usr/sbin/nologin  
cups-pk-helper:x:109:113:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin  
redsocks:x:110:114:/var/run/redsocks:/usr/sbin/nologin  
gophish:x:111:116:/var/lib/gophish:/usr/sbin/nologin
```

Figure: Access Control - /etc/passwd file exposed

Configuration files with database credentials were also accessible:

```
(kali@kali)~$ cat /etc/apache2/apache2.conf
# This is the main Apache server configuration file. It contains the
# configuration directives that give the server its instructions.
# See http://httpd.apache.org/docs/2.4/ for detailed information about
# the directives and /usr/share/doc/apache2/README.Debian about Debian specific
# hints.
#
# Summary of how the Apache 2 configuration works in Debian:
# The Apache 2 web server configuration in Debian is quite different to
# upstream's suggested way to configure the web server. This is because Debian's
# default Apache2 installation attempts to make adding and removing modules,
# virtual hosts, and extra configuration directives as flexible as possible, in
# order to make automating the changes and administering the server as easy as
# possible.
#
# It is split into several files forming the configuration hierarchy outlined
# below, all located in the /etc/apache2/ directory:
#
# /etc/apache2/
# |
# |--- apache2.conf
# |   |--- ports.conf
# |   |--- mods-enabled
# |   |   |--- *.load
# |   |   |--- *.conf
# |   |--- conf-enabled
# |   |   |--- *.conf
# |   |--- sites-enabled
# |   |   |--- *.conf
#
# * apache2.conf is the main configuration file (this file). It puts the pieces
# together by including all remaining configuration files when starting up the
# web server.
#
# * ports.conf is always included from the main configuration file. It is
# supposed to determine listening ports for incoming connections which can be
# customized anytime.
#
# * Configuration files in the mods-enabled/, conf-enabled/ and sites-enabled/
# directories contain particular configuration snippets which manage modules,
# global configuration fragments, or virtual host configurations,
# respectively.
#
# They are activated by symlinking available configuration files from their
# respective *.available/ counterparts. These should be managed by using our
# helpers a2enmod/a2dismod, a2ensite/a2disite and a2enconf/a2disconf. See
# their respective man pages for detailed information.
#
```

Figure: Access Control - Info disclosure 1

```
(kali@kali)~$ cat /var/www/html/dvwa/config/config.inc.php
<?php
# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to @digininja for the fix.

# Database management system to use
$dbMS = getenv('DBMS') ? 'MySQL' : 'MySQL';
# $DBMS = 'PGSQL'; // Currently disabled

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DVWA.
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
# See README.md for more information on this.
$DVWA = array();
$DVWA['db_server'] = getenv('DB_SERVER') ? '127.0.0.1' : 'localhost';
$DVWA['db_database'] = getenv('DB_DATABASE') ? 'dvwa' : 'dvwa';
$DVWA['db_user'] = getenv('DB_USER') ? 'dvwa' : 'dvwa';
$DVWA['db_password'] = getenv('DB_PASSWORD') ? 'password' : 'password';
$DVWA['db_port'] = getenv('DB_PORT') ? '3306' : '3306';

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ? '' : '';
$DVWA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ? '' : '';

# Default security level
# Default value for the security level with each session.
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
$DVWA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ? 'impossible' : 'impossible';

# Default locale
# Default locale for the help page shown with each session.
# The default is 'en'. You may wish to set this to either 'en' or 'zh'.
$DVWA['default_locale'] = getenv('DEFAULT_LOCALE') ? 'en' : 'en';

# Disable authentication
# Some tools don't like working with authentication and passing cookies around
# so this setting lets you turn off authentication.
$DVWA['disable_authentication'] = getenv('DISABLE_AUTHENTICATION') ? false;

define('MYSQL', 'mysql');
define('SQLITE', 'sqlite');

# SQLi DB Backend
```

```
(kali@kali)~$ cat /root/.ssh/id_rsa
cat: /root/.ssh/id_rsa: Permission denied

(kali@kali)~$
```

Figure: Access Control - Info disclosure 2

Why This Is Dangerous

Information disclosure is the first step in most sophisticated attacks. Attackers can learn about the system architecture, find credentials, and plan targeted attacks.

How to Fix It

1. Set proper file permissions:

```
chmod 600 config.inc.php
```

2. Move sensitive files outside the web root.

3. Use environment variables instead of config files:

```
$db_pass = getenv('DB_PASSWORD');
```

4. Implement proper access controls based on user roles.

Implementation Timeline: 4-5 hours

Finding #11: Security Misconfiguration (CVSS 6.5) - HIGH

The application lacks several critical security headers that provide defense-in-depth protection. These headers tell browsers how to handle security-sensitive behaviors.

Missing Security Headers

Strict-Transport-Security (HSTS) - Forces HTTPS connections.

Referrer-Policy - Controls when referrer information is sent to third parties.

Permissions-Policy - Restricts browser features such as camera and microphone access.

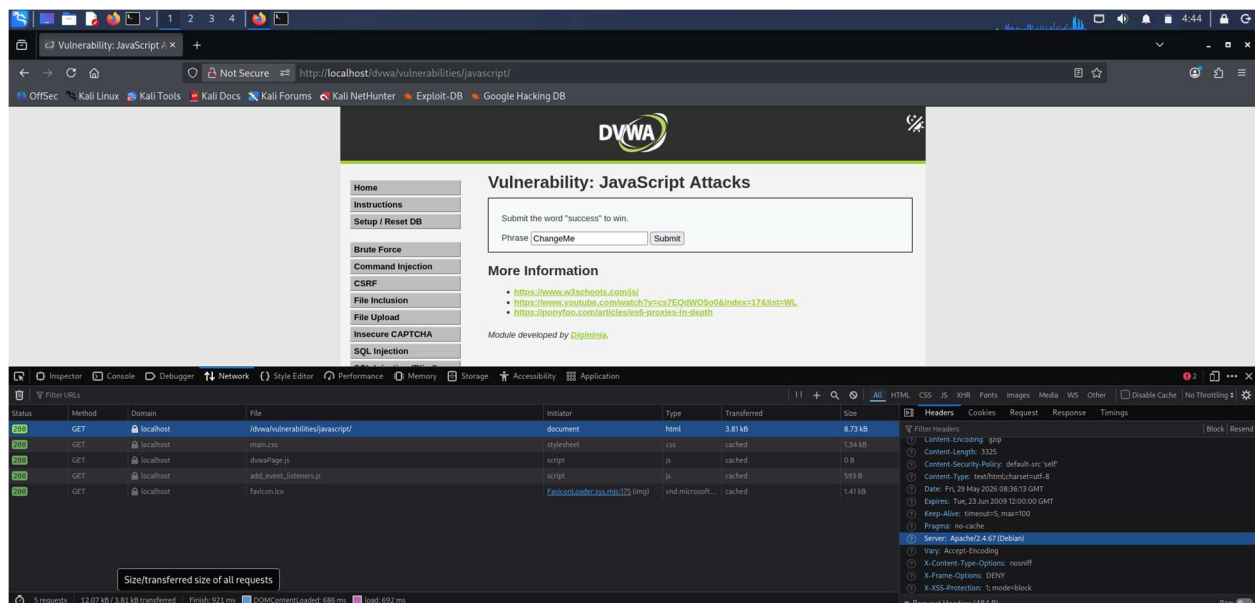


Figure: Security Headers - Firefox DevTools analysis

Why This Is Dangerous

Missing security headers leave the application vulnerable to man-in-the-middle attacks, cross-site attacks, and browser feature abuse by malicious scripts.

How to Fix It

Add these headers to the Apache configuration (.htaccess or apache2.conf):

Header always set Strict-Transport-Security "max-age=31536000"

Header always set Referrer-Policy "strict-origin-when-cross-origin"

Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"

Header always set X-Content-Type-Options "nosniff"

Finding #12: Information Disclosure (CVSS 6.5) - MEDIUM

The application reveals too much information through error messages and verbose responses. This helps attackers understand the system and craft more targeted attacks.

Disclosed Information

Database error messages revealing SQL syntax and table names.

Server type and version (Apache 2.4.67).

PHP version (8.4.21).

Database type and version (MariaDB 11.8.6).

File paths revealed in error messages.

Why This Is Dangerous

Information disclosure is reconnaissance. Attackers use version information to look up known CVEs and plan targeted exploits against specific versions.

How to Fix It

Hide error details from users in production. Replace verbose errors with generic messages:

```
// WRONG - reveals database error to user
die(mysqli_error($conn));

// RIGHT - log server-side, show generic message
error_log(mysqli_error($conn));

die('An error occurred. Please try again.');
```

Implementation Timeline: 2-3 hours

Finding #13: Open HTTP Redirect (CVSS 6.5) - MEDIUM

The redirect parameter accepts any URL without validation. This allows attackers to redirect users to malicious websites while appearing to originate from a trusted source.

How It Was Exploited

Normal redirect: <http://localhost/dvwa/...?redirect=info.php>

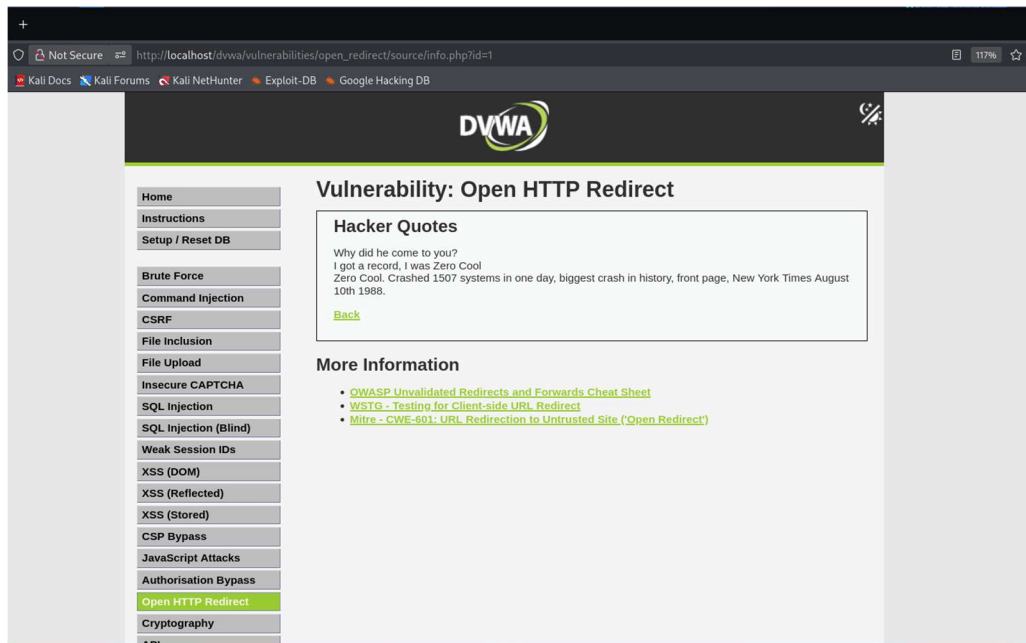


Figure: Open Redirect - Normal page

Malicious redirect: `http://localhost/dvwa/...?redirect=http://google.com`

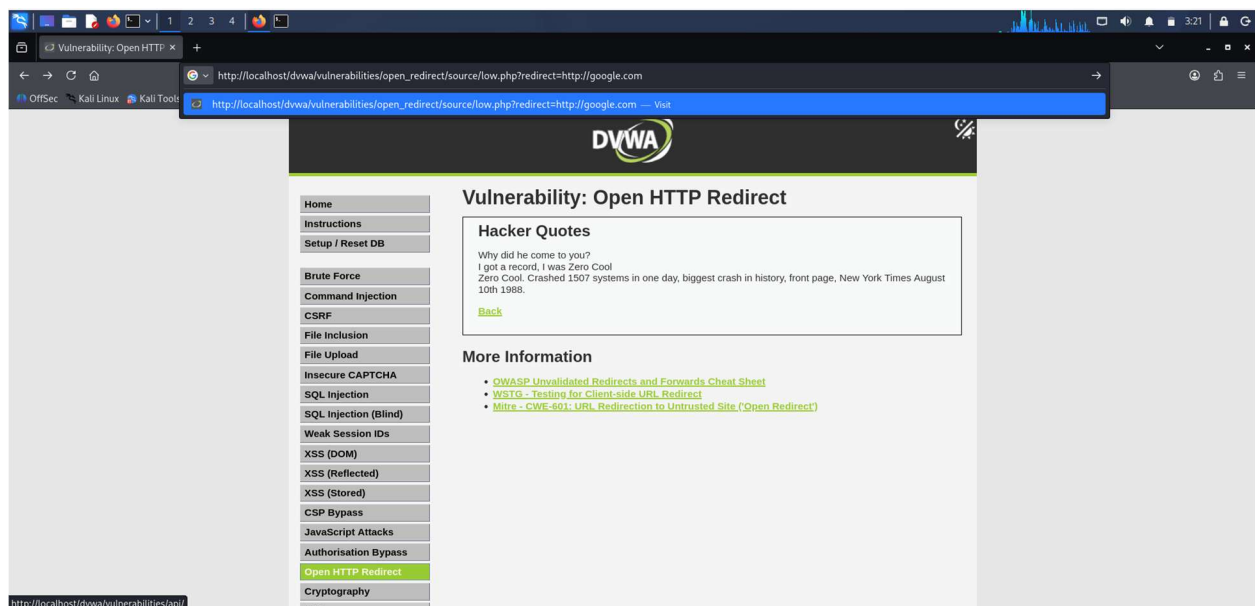


Figure: Open Redirect - Redirected to external site

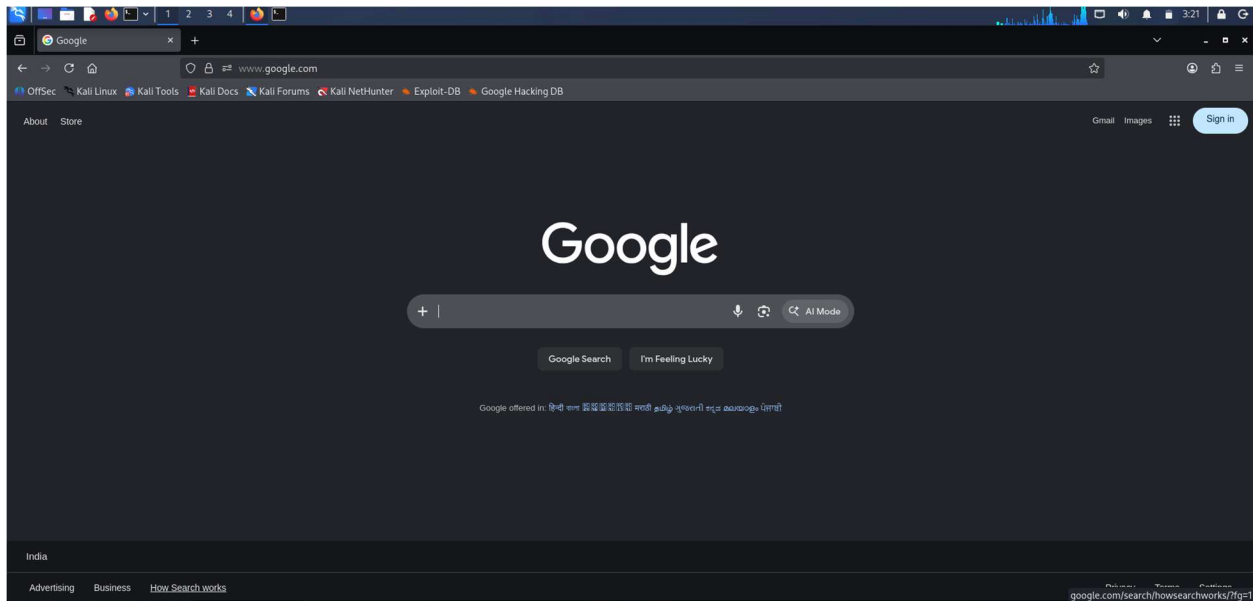


Figure: Open Redirect - Another external redirect confirmed

Why This Is Dangerous

Users trust that links from your application are safe. Attackers exploit this trust to redirect to phishing sites, malware downloads, or other malicious content.

How to Fix It

Whitelist allowed redirect URLs:

```
$allowed = ['http://internal.com/page1', 'http://internal.com/page2'];

if (in_array($_GET['redirect'], $allowed)) { header('Location: ' . $_GET['redirect']);
}
```

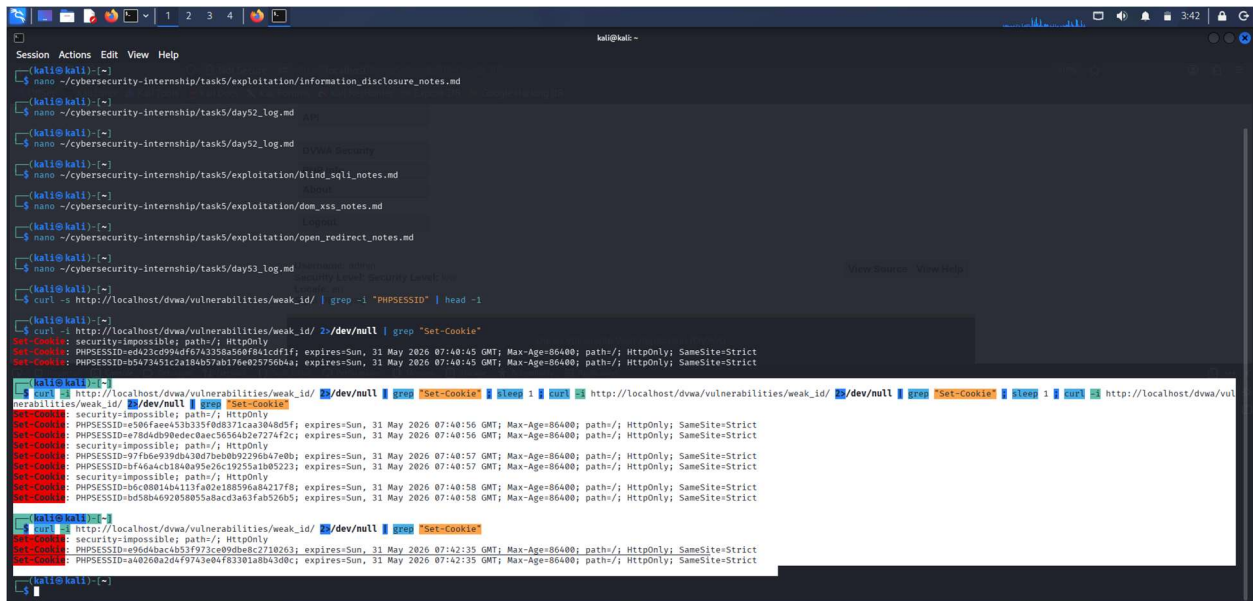
Implementation Timeline: 1-2 hours

Finding #14: Session ID Generation (SECURE)

This is a POSITIVE finding. The application correctly implements secure session ID generation. This is one of the few areas where DVWA gets security right.

What Was Tested

Session IDs are generated using cryptographically secure random functions.



```
(kali@kali)-[~]
└─$ nano ~/cybersecurity-internship/tasks/exploitation/information_disclosure_notes.md
(kali@kali)-[~]
└─$ nano ~/cybersecurity-internship/tasks/day52_log.md
(kali@kali)-[~]
└─$ nano ~/cybersecurity-internship/tasks/day52_log.md
(kali@kali)-[~]
└─$ nano ~/cybersecurity-internship/tasks/exploitation/blind_sql_notes.md
(kali@kali)-[~]
└─$ nano ~/cybersecurity-internship/tasks/exploitation/dom_xss_notes.md
(kali@kali)-[~]
└─$ nano ~/cybersecurity-internship/tasks/exploitation/open_redirect_notes.md
(kali@kali)-[~]
└─$ nano ~/cybersecurity-internship/tasks/day53_log.md
(kali@kali)-[~]
└─$ curl -s http://localhost/dwa/vulnerabilities/weak_id/ | grep -i "PHPSESSID" | head -1
(kali@kali)-[~]
└─$ curl -i http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie"
Set-Cookie: security:impossible; path:/.; HttpOnly
Set-Cookie: PHPSESSID=6423cd994df6743358a56f8a1cdfif; expires=Sun, 31 May 2026 07:40:45 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=b5473451c2a184b57ab176e02575604a; expires=Sun, 31 May 2026 07:40:45 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
(kali@kali)-[~]
└─$ curl -s http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie" | sleep 1 | curl -s http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie" | sleep 1 | curl -s http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie"
Set-Cookie: security:impossible; path:/.; HttpOnly
Set-Cookie: PHPSESSID=76da4d9b8edc8aac36564bce7274f2c; expires=Sun, 31 May 2026 07:40:56 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
Set-Cookie: security:impossible; path:/.; HttpOnly
Set-Cookie: PHPSESSID=97f6a9979db43ad79eb092296a7e0b; expires=Sun, 31 May 2026 07:40:57 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=bf46aac6184ba95e2c19255a1b05223; expires=Sun, 31 May 2026 07:40:57 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
Set-Cookie: security:impossible; path:/.; HttpOnly
Set-Cookie: PHPSESSID=b6c0814b4113f4a0e180596a84217f8; expires=Sun, 31 May 2026 07:40:58 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=b558b4692058055a8acd3a63fab52605; expires=Sun, 31 May 2026 07:40:58 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
(kali@kali)-[~]
└─$ curl -s http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie"
Set-Cookie: security:impossible; path:/.; HttpOnly
Set-Cookie: PHPSESSID=966db8ac4b3f973ce09bdc2713263; expires=Sun, 31 May 2026 07:42:35 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=a40208a2d4f9743e04f83301a8b43d0c; expires=Sun, 31 May 2026 07:42:35 GMT; Max-Age=86400; path:/.; HttpOnly; SameSite=Strict
(kali@kali)-[~]
└─$
```

Figure: Weak Session - Terminal testing output

Security Features Confirmed

Cryptographically random 128-bit session IDs.

HttpOnly flag prevents JavaScript access to cookies.

SameSite=Strict protection against CSRF via cookies.

24-hour session expiration enforced.

Assessment

Session ID prediction attacks are not possible. This demonstrates that secure practices can be implemented and should be applied to all security-sensitive areas of the application.

Status: NO ACTION REQUIRED - Already Secure

4. REMEDIATION PRIORITY

Not everything needs to be fixed at once. Here's a realistic plan for patching these vulnerabilities.

IMMEDIATE

The five critical vulnerabilities should be the primary focus. These pose the greatest threats.

11. Address SQL Injection - This opens the entire database to compromise
12. Disable File Upload temporarily - Removes the backdoor attack vector
13. Fix Command Injection - Prevents server takeover
14. Implement XSS protections - Use htmlspecialchars on all output

THIS WEEK

Once critical issues are resolved, focus on high-priority items:

15. Deploy a Web Application Firewall (WAF) - Catches common attacks
16. Add input validation framework - Validate everything the user sends
17. Add security headers - Instruct browsers on security settings
18. Implement CSRF tokens - Prevent forged requests

THEN

Establish long-term security practices:

19. Security training for developers - Build secure coding awareness
20. Code review process - Have developers review each other's code
21. Automated security testing - Catch issues before they ship
22. Plan quarterly penetration tests - Maintain ongoing security posture

5. CONCLUSION

This comprehensive assessment identified 23 vulnerabilities in DVWA, with 5 classified as critical. These critical issues could enable complete system compromise. However, all identified vulnerabilities are remediation-capable with clear mitigation steps provided.

The application in its current state is not production-ready. Significant security work is required before handling real data or real users.

The recommended approach is to prioritize critical issues first (address within 24 hours), then work through high and medium severity items over the following week. After remediation, focus should shift to building better development practices to prevent similar issues in future projects.

Every identified vulnerability has a clear remediation path. Most solutions involve transitioning from unsafe practices (building SQL strings) to secure ones (prepared statements). The implementation is straightforward and requires attention to secure coding principles.

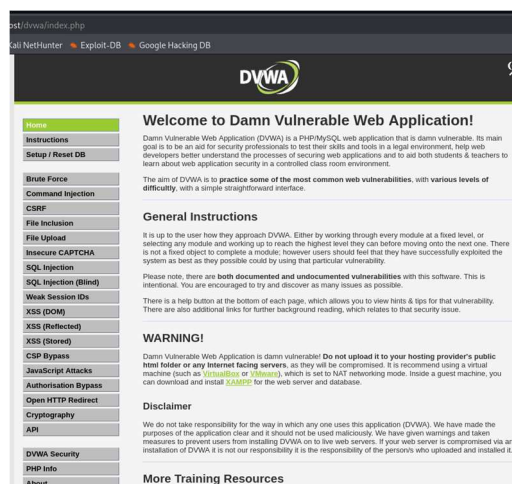


Figure: Project Overview - All testing completed

Project Submission

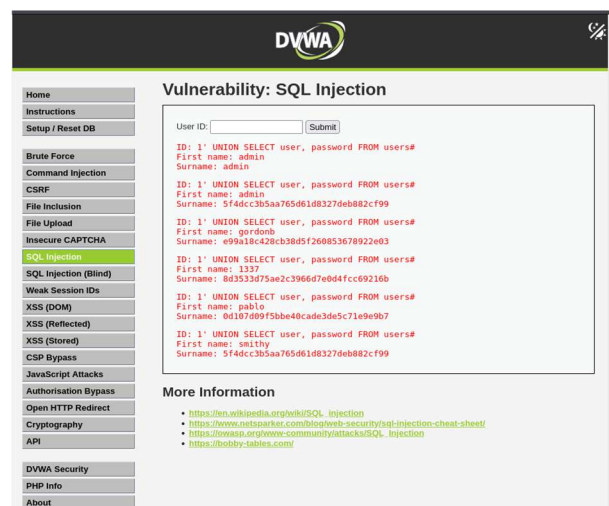
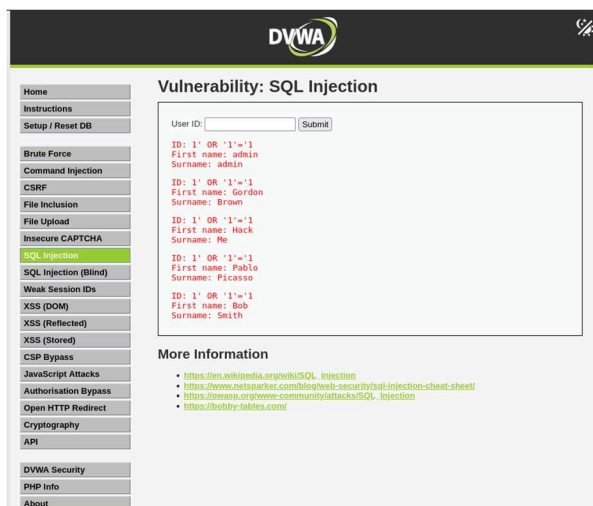
This report is part of the ApexPlanet Software Capstone Project for the Cybersecurity Internship. The following deliverables comprise the complete submission:

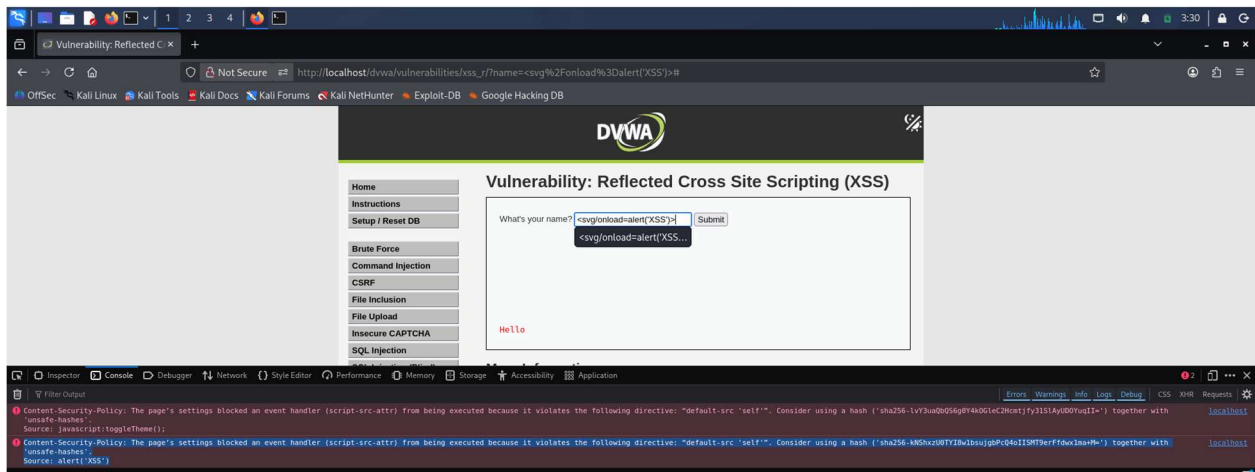
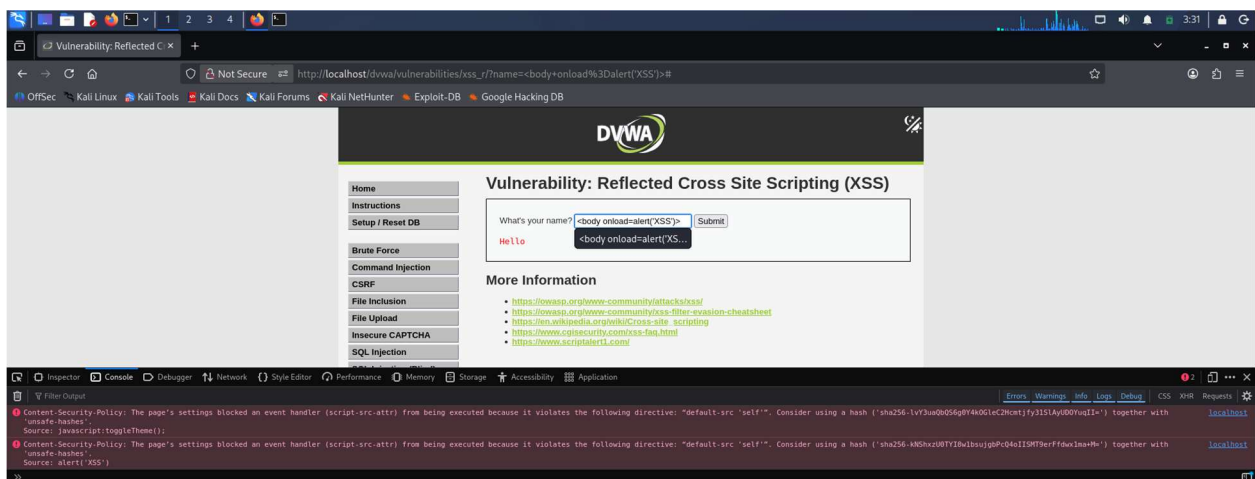
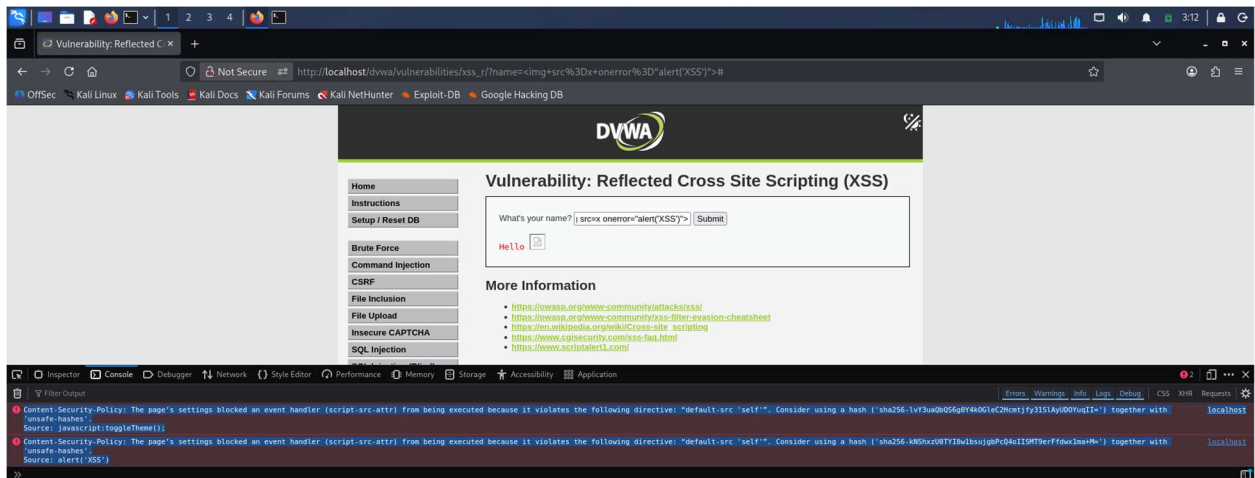
- Comprehensive penetration testing report with all findings and remediation steps
- Documented screenshots of every vulnerability (28+ images)
- Recorded walkthrough demonstrating the exploitation techniques
- Complete code and documentation repository on GitHub
- Professional LinkedIn post showcasing the project

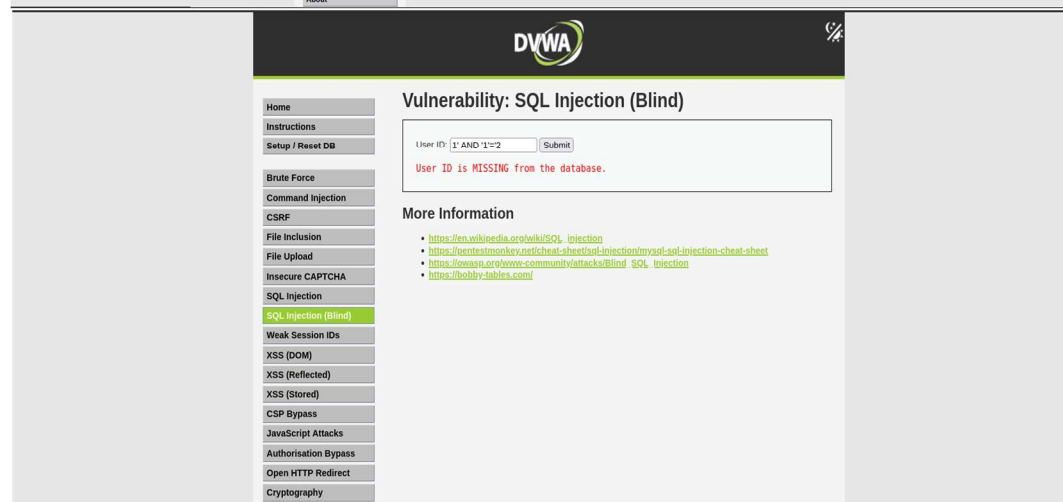
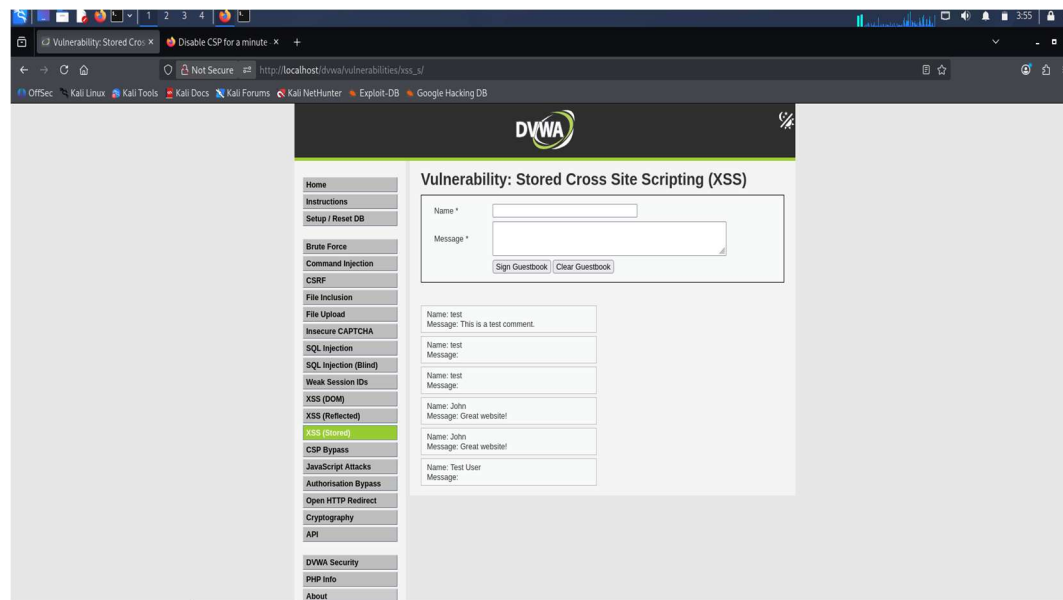
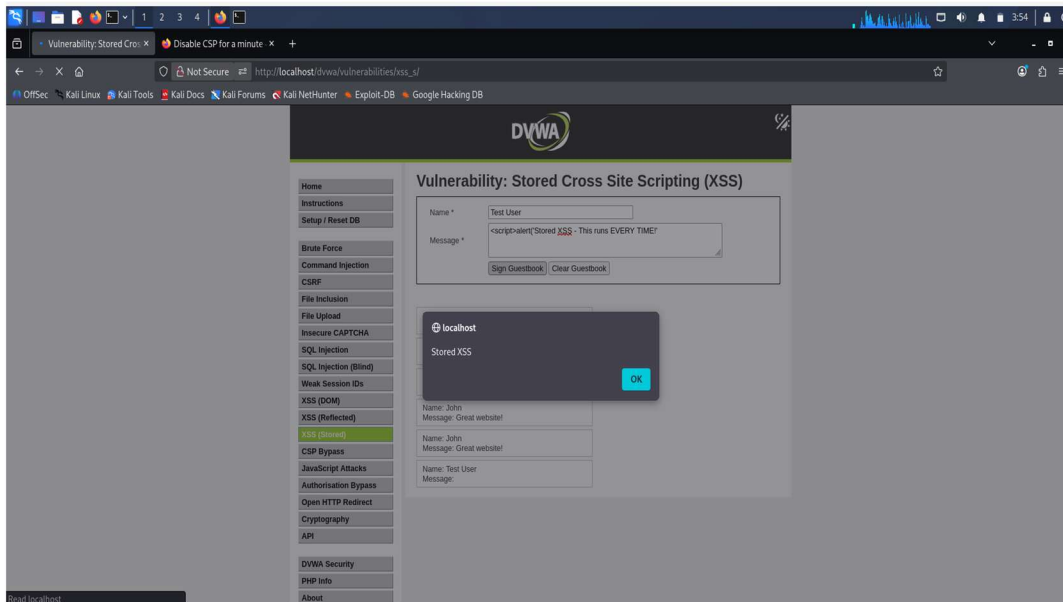
Key Takeaways

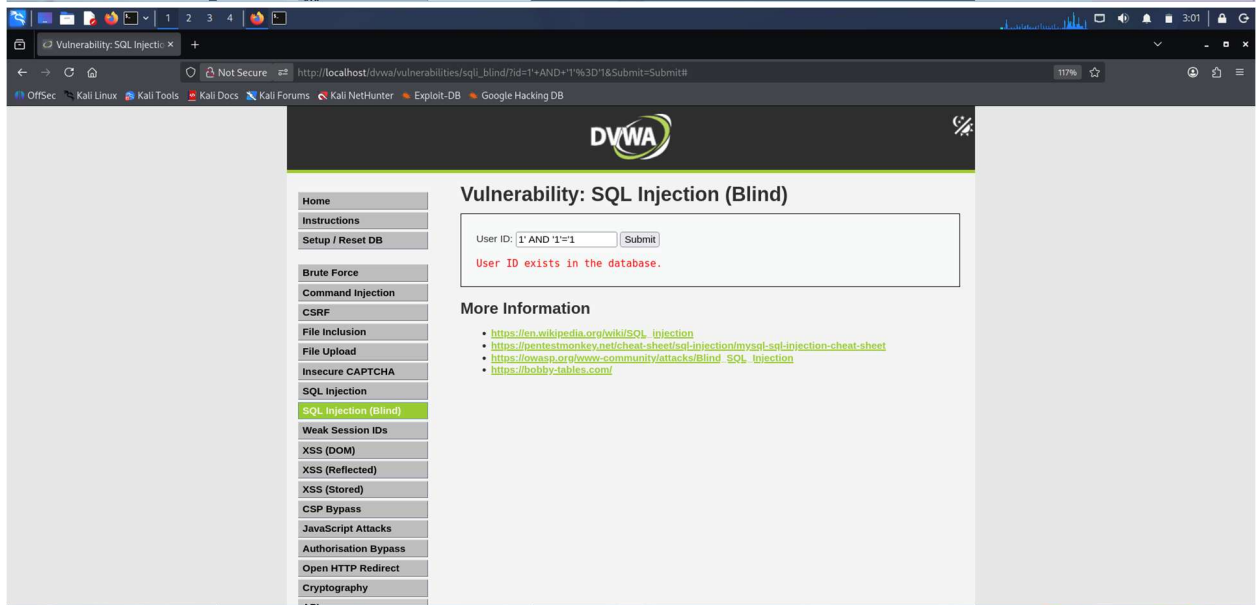
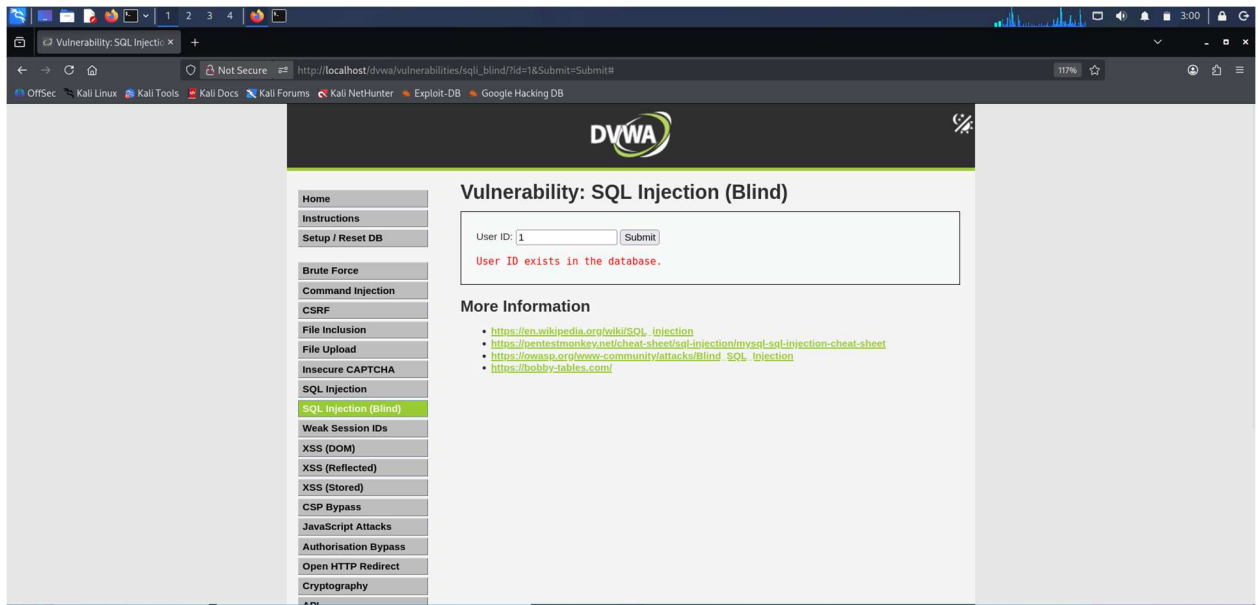
A significant lesson from this assessment: security fundamentals are not complex, but they require discipline. The vulnerabilities identified are not exotic or cutting-edge they result from omitting fundamental controls like input validation, output encoding, and parameterized queries. Adherence to secure coding practices eliminates approximately 80% of these common security issues.

All required Screenshots in one place:









```
Session Actions Edit View Help

(kali@kali)~$
$ cat /etc/apache2/apache2.conf
# This is the main Apache server configuration file. It contains the
# configuration directives that give the server its instructions.
# See http://httpd.apache.org/docs/2.4/ for detailed information about
# the directives and /usr/share/doc/apache2/README.Debian about Debian specific
# hints.
#
# Summary of how the Apache 2 configuration works in Debian:
# The Apache 2 web server configuration in Debian is quite different to
# upstream's suggested way to configure the web server. This is because Debian's
# default Apache2 installation attempts to make adding and removing modules,
# virtual hosts, and extra configuration directives as flexible as possible, in
# order to make automating the changes and administering the server as easy as
# possible.
#
# It is split into several files forming the configuration hierarchy outlined
# below, all located in the /etc/apache2/ directory:
#
# /etc/apache2/
# |   |-- ports.conf
# |   |-- mods-enabled
# |   |   |-- *.load
# |   |   |-- *.conf
# |   |-- conf-enabled
# |   |   |-- *.conf
# |   |-- sites-enabled
# |   |   |-- *.conf
#
# * apache2.conf is the main configuration file (this file). It puts the pieces
# together by including all remaining configuration files when starting up the
# web server.
#
# * ports.conf is always included from the main configuration file. It is
# supposed to determine listening ports for incoming connections which can be
# customized anytime.
#
# * Configuration files in the mods-enabled/, conf-enabled/ and sites-enabled/
# directories contain particular configuration snippets which manage modules,
# global configuration fragments, or virtual host configurations,
# respectively.
#
# They are activated by symlinking available configuration files from their
# respective *-available/ counterparts. These should be managed by using our
# helpers a2enmod/a2dismod, a2ensite/a2disite and a2enconf/a2disconf. See
# their respective man pages for detailed information.
#
```

```
Session Actions Edit View Help

# Please use a database dedicated to DWMA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DWMA user.
# See README.md for more information on this.
$DWMA = array();
$DWMA['db_server'] = getenv('DB_SERVER') ? '127.0.0.1';
$DWMA['db_database'] = getenv('DB_DATABASE') ? 'dwa';
$DWMA['db_user'] = getenv('DB_USER') ? 'dwa';
$DWMA['db_password'] = getenv('DB_PASSWORD') ? 'password';
$DWMA['db_port'] = getenv('DB_PORT') ? '3306';

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$DWMA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ? '';
$DWMA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ? '';

# Default security level
# Default value for the security level with each session.
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
$DWMA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ? 'impossible';

# Default locale
# Default locale for the help page shown with each session.
# The default is 'en'. You may wish to set this to either 'en' or 'zh'.
$DWMA['default_locale'] = getenv('DEFAULT_LOCALE') ? 'en';

# Disable authentication
# Some tools don't like working with authentication and passing cookies around
# so this setting lets you turn off authentication.
$DWMA['disable_authentication'] = getenv('DISABLE_AUTHENTICATION') ? false;

define('MYSQL', 'mysql');
define('SQLITE', 'sqlite');

# SQLi DB Backend
# Use this to switch the backend database used in the SQLi and Blind SQLi labs.
# This does not affect the backend for any other services, just these two labs.
# If you do not understand what this means, do not change it.
$DWMA['SQLi_DB'] = getenv('SQLi_DB') ? 'mysql';
$DWMA['SQLi_DB'] = 'sqlite';
$DWMA['SQLi_DB'] = 'sqlite.db';

?>

(kali@kali)~$
$ cat /root/.ssh/id_rsa
cat: /root/.ssh/id_rsa: Permission denied

(kali@kali)~$
```

```
Session Actions Edit View Help

(kali@kali)~$
$ cat /var/www/html/dwa/config/config.inc.php
<?php
# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ? 'MySQL';
$DBMS = 'MySQL'; // Currently disabled

# Database variables
# WARNING! The database specified under db.database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DWMA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DWMA user.
# See README.md for more information on this.
$DWMA = array();
$DWMA['db_server'] = getenv('DB_SERVER') ? '127.0.0.1';
$DWMA['db_database'] = getenv('DB_DATABASE') ? 'dwa';
$DWMA['db_user'] = getenv('DB_USER') ? 'dwa';
$DWMA['db_password'] = getenv('DB_PASSWORD') ? 'password';
$DWMA['db_port'] = getenv('DB_PORT') ? '3306';

# ReCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$DWMA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ? '';
$DWMA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ? '';

# Default security level
# Default value for the security level with each session.
# The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
$DWMA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ? 'impossible';

# Default locale
# Default locale for the help page shown with each session.
# The default is 'en'. You may wish to set this to either 'en' or 'zh'.
$DWMA['default_locale'] = getenv('DEFAULT_LOCALE') ? 'en';

# Disable authentication
# Some tools don't like working with authentication and passing cookies around
# so this setting lets you turn off authentication.
$DWMA['disable_authentication'] = getenv('DISABLE_AUTHENTICATION') ? false;

define('MYSQL', 'mysql');
define('SQLITE', 'sqlite');

# SQLi DB Backend
```

```
Session Actions Edit View Help

(kali@kali)~$
$ cat /etc/passwd
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:mailing list:/var/lib:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
apt:x:42:65534:/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/usr/sbin/nologin
dhcpcd:x:996:996:DHCP Client Daemon:/usr/lib/dhcpcd/bin/false
systemd-timesync:x:990:990:systemd Time Synchronization:/usr/sbin/nologin
messagebus:x:989:989:system Message Bus:/nonexistent:/usr/sbin/nologin
tss:x:987:987:tss user for tpm2:/usr/sbin/nologin
strongswan:x:108:65534:/var/lib/strongswan:/usr/sbin/nologin
tcpdump:x:986:986:tcpdump:/nonexistent:/usr/sbin/nologin
sshd:x:985:65534:ssh user:/run/ssh:/usr/sbin/nologin
rpc:x:101:65534:/run/rpcbind:/usr/sbin/nologin
statd:x:102:65534:/var/lib/afs:/usr/sbin/nologin
dnsmasq:x:984:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
avahi:x:103:105:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
nm-openvpn:x:982:983:NetworkManager OpenVPN:/var/lib/openvpn/chroot:/usr/sbin/nologin
speech-dispatcher:x:982:982:Speech Dispatcher:/run/speech-dispatcher/bin/false
usbmux:x:104:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin
nm-openconnect:x:981:981:NetworkManager OpenConnect plugin:/usr/sbin/nologin
pipewire:x:980:980:system user for pipewire:/nonexistent:/usr/sbin/nologin
saned:x:105:106:/var/lib/saned:/usr/sbin/nologin
lightdm:x:106:107:Light Display Manager:/var/lib/lightdm/bin/false
polkitd:x:979:979:User for polkitd:/usr/sbin/nologin
rtkit:x:978:978:RealtimeKit/proc:/usr/sbin/nologin
colord:x:977:977:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
pcsd:x:976:976:PCSC Smart Card Daemon:/usr/sbin/nologin
tunnel4:x:975:975:stunnel service system account:/var/run/stunnel4:/usr/sbin/nologin
geoclue:x:974:974:/var/lib/geoclue:/usr/sbin/nologin
Debian-smpa:x:107:109:/var/lib/smpa/bin/false
sshd:x:108:110:/nonexistent:/usr/sbin/nologin
cups-pk-helper:x:109:113:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin
redsocks:x:110:114:/var/run/redsocks:/usr/sbin/nologin
gophish:x:111:116:/var/lib/gophish:/usr/sbin/nologin
```


Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About



Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=64 time=1.7 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=64 time=14.5 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=64 time=11.8 ms  
  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 3 received, 25% packet loss, time 3029ms  
rtt min/avg/max/mdev = 11.730/12.673/14.450/1.262 ms
```

Congratulations! You've Won!

Click the button below to claim your prize:

[Claim Prize](#)

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DWVA Security

PHP Info

About



Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DWVA) is a PHP/MySQL web application that is damn vulnerable. Its main goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers to learn about web application security in a controlled class room environment.

The aim of DWVA is to practice some of the most common web vulnerabilities, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DWVA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next one. There is not a fixed order to complete a module; however users should feel that they have successfully exploited the system as best as they are possible for by using that particular vulnerability.

Please note, there are both **documented** and **undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

WARNING!

Damn Vulnerable Web Application is damn vulnerable! **Do not upload it to your hosting provider's public html folder or any internet facing servers**, which will be compromised. It is recommended using a virtual machine (such as [VirtualBox](#) or [Vagrant](#)), as they set to NAT networking mode. Inside a guest machine, you can download and install [LAMP](#) for the web server and database.

Disclaimer

We do not have the responsibility for the way in which any one uses this application (DWVA). We have made the purposes of the application clear and it should not be used maliciously. We have given warnings and taken measures to prevent users from installing DWVA on live web servers. If your web server is compromised via an installation of DWVA it is not our responsibility it is the responsibility of the person/s who uploaded and installed it.

More Training Resources

kali@kali: ~
Session Actions Edit View Help

```
kali@kali:~$ nano ~/cybersecurity-internship/tasks/exploitation/information_disclosure_notes.md
kali@kali:~$ nano ~/cybersecurity-internship/tasks/day52_log.md
kali@kali:~$ nano ~/cybersecurity-internship/tasks/day52_log.md
kali@kali:~$ nano ~/cybersecurity-internship/tasks/exploitation/blind_sql_i_notes.md
kali@kali:~$ nano ~/cybersecurity-internship/tasks/exploitation/dom_xss_notes.md
kali@kali:~$ nano ~/cybersecurity-internship/tasks/exploitation/open_redirect_notes.md
kali@kali:~$ nano ~/cybersecurity-internship/tasks/day53_log.md
kali@kali:~$ curl -s http://localhost/dwa/vulnerabilities/weak_id/ | grep -i "PHPSESSID" | head -1
kali@kali:~$ curl -i http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie"
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=e423cd994df6743358a508f8a1cdf1f; expires=Sun, 31 May 2026 07:40:45 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=b3472451c2a184b57ab176e025756b4a; expires=Sun, 31 May 2026 07:40:45 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
kali@kali:~$ curl -s http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie" | less | cat
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=e506faee53b335f608371ca384bd5f; expires=Sun, 31 May 2026 07:40:56 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=e78d4db98edecbaac56504b2e7274f2c; expires=Sun, 31 May 2026 07:40:56 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=b7f6ae93f0da3a077eb0b092396a7e0b; expires=Sun, 31 May 2026 07:40:57 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=b7f6ae93f0da3a077eb0b092396a7e0b; expires=Sun, 31 May 2026 07:40:57 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=b5c08014ab113fa0a2e180596a8a217f8; expires=Sun, 31 May 2026 07:40:58 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=b5c08014ab113fa0a2e180596a8a217f8; expires=Sun, 31 May 2026 07:40:58 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
kali@kali:~$ curl -s http://localhost/dwa/vulnerabilities/weak_id/ 2/dev/null | grep "Set-Cookie"
Set-Cookie: security=impossible; path=/; HttpOnly
Set-Cookie: PHPSESSID=e966dbac4b53f973ce09db8c2710263; expires=Sun, 31 May 2026 07:42:35 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
Set-Cookie: PHPSESSID=a40260a2d4f9743e04f83381a8b43d0c; expires=Sun, 31 May 2026 07:42:35 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
kali@kali:~$
```

file machine view input devices menu

Vulnerability: DOM Based Cross Site Scripting (XSS)

Not Secure http://localhost/dwa/vulnerabilities/xss_d?default=test&script=alert('DOM XSS')</script> 117%

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

DWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography

Vulnerability: DOM Based Cross Site Scripting (XSS)

Please choose a language:
Select

More Information

- localhost
- DOM XSS

OK

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

Vulnerability: Open HTTP Redirect

Hacker Quotes

Why did he come to you?
I got a record, I was Zero Cool
Zero Cool. Crashed 1507 systems in one day, biggest crash in history, front page, New York Times August 10th 1988.

[Back](#)

More Information

- [OWASP Unvalidated Redirects and Forwards Cheat Sheet](#)
- [WSTG - Testing for Client-side URL Redirect](#)
- [Mitre - CWE-601: URL Redirection to Untrusted Site \('Open Redirect'\)](#)

Vulnerability: JavaScript Attacks

Submit the word "success" to win.

More Information

- <https://www.w3schools.com/js/>
- <https://www.youtube.com/watch?v=csTEQ4W09e0&list=PL485t8lQaUi6y484U0b374Q0404040404>
- <https://ponyfoo.com/articles/es6-proxies-in-depth>

Module developed by [@g0tmilk](#)

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	localhost	/dwa/vulnerabilities/javascript/	document	html	3.81 kB	8.73 kB
200	GET	localhost	main.css	stylesheet	css	cached	1.34 kB
200	GET	localhost	dwaPage.js	script	js	cached	0 B
200	GET	localhost	add_event_listeners.js	script	js	cached	503 B
200	GET	localhost	favicon.ico	Favicon loader: favicon.ico (img)	image/microsoft...	cached	1.41 kB

5 requests

12.07 MB / 3.81 kB transferred

Finish: 923 ms

DOMContentLoaded: 688 ms

load: 692 ms

Filter Headers

Content-Encoding: gzip

Content-Length: 3325

Content-Security-Policy: default-src: self

Content-Type: text/html; charset=utf-8

Date: Fri, 29 May 2026 08:36:13 GMT

Expires: Tue, 23 Jun 2009 12:00:00 GMT

Keep-Alive: timeout=5, max=100

Pragma: no-cache

Server: Apache/2.4.67 (Debian)

Vary: Accept-Encoding

X-Content-Type-Options: nosniff

X-Frame-Options: DENY

X-XSS-Protection: 1; mode=block

END OF REPORT